

Aula 01

Componente Curricular: Sistemas Web II

1. Objetivos

- Compreender a diferença entre persistir dados em uma estrutura em memória e persistir em um banco de dados real.
- Configurar a conexão de uma aplicação FastAPI com um banco de dados MySQL utilizando o SQLAlchemy como ORM.
- Modelar, em Python, a tabela do banco de dados a partir da modelagem do recurso REST.
- Implementar os primeiros endpoints (GET e POST) persistindo e consultando dados reais no MySQL.

2. Conteúdo programático

- Revisão: recurso, endpoint e métodos **HTTP** em uma API REST.
- O que é um **ORM** (Object-Relational Mapping) e por que utilizá-lo em vez de escrever SQL manualmente.
- Criação do banco de dados no MySQL.
- Configuração da conexão com **SQLAlchemy** (engine, sessão, base declarativa).
- Criação do modelo **ORM** (tabela) e do **schema Pydantic** (validação de entrada/saída de dados).
- Implementação de **GET** e **POST** utilizando uma sessão real do banco de dados.

3. Recursos necessários

- Computador com Python 3 e MySQL Server instalados (localmente ou em container).
- Ferramenta de administração do MySQL (MySQL Workbench ou cliente de linha de comando).
- Editor de código (VS Code).
- Navegador para acessar a documentação interativa do FastAPI (Swagger UI em /docs).

4. Desenvolvimento

4.1 Abertura

Apresentação do 3º bimestre e explicação de que, a partir desta aula, os dados passam a ser persistidos de verdade em um banco de dados MySQL — diferentemente de uma solução com lista em memória, que perde todos os dados a cada reinício do servidor. Retomar a modelagem do recurso Produto (id, nome, preço, quantidade) que servirá de base para a tabela do banco.

4.2 Criando o banco de dados no MySQL

Criar o banco de dados que armazenará as tabelas da aplicação:

```
-- Executar no MySQL Workbench ou no cliente de linha de comando (mysql -u root -p)
CREATE DATABASE loja;
```

Apenas o banco de dados (**schema**) é criado manualmente; as tabelas serão criadas automaticamente pelo **SQLAlchemy**, a partir dos modelos definidos em Python.

4.3 Instalando as bibliotecas Python

```
# Ambiente virtual já criado nas aulas anteriores
pip install fastapi uvicorn sqlalchemy pymysql
```

O papel de cada biblioteca: SQLAlchemy é o ORM utilizado para mapear classes Python em tabelas do banco; PyMySQL é o driver que permite ao SQLAlchemy se conectar a um banco MySQL.

4.4 Configurando a conexão com o banco (database.py)

Criar o arquivo responsável por configurar a conexão com o MySQL:

```
# database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

# Formato: mysql+pymysql://usuario:senha@host/nome_do_banco
DATABASE_URL = 'mysql+pymysql://root:senha@localhost/loja'

engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()
```

```
# Função de dependência: abre uma sessão por requisição e garante o fechamento
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Explicação de cada peça: **engine** representa a conexão com o banco; **SessionLocal** cria uma nova sessão de trabalho a cada requisição; **Base** é a classe da qual todos os modelos ORM herdam; **get_db** é a função que o FastAPI usará como dependência para fornecer (e depois fechar) uma sessão em cada endpoint.

4.5 Modelando a tabela Produto (models.py)

Criar o modelo ORM que representa a tabela produtos no banco de dados:

```
# models.py
from sqlalchemy import Column, Integer, String, Float
from database import Base

class ProdutoDB(Base):
    __tablename__ = 'produtos'

    id = Column(Integer, primary_key=True, index=True)
    nome = Column(String(100), nullable=False)
    preco = Column(Float, nullable=False)
    quantidade = Column(Integer, nullable=False)
```

4.6 Schemas de entrada e saída (schemas.py)

Criar os schemas Pydantic que definem o formato dos dados trocados pela API — diferentes do modelo ORM, que representa a tabela do banco:

```
# schemas.py
from pydantic import BaseModel

class ProdutoBase(BaseModel):
```

```
nome: str
preco: float
quantidade: int

class ProdutoCreate(ProdutoBase):
    pass

class ProdutoResponse(ProdutoBase):
    id: int

class Config:
    from_attributes = True
```

Diferença entre o modelo ORM (ProdutoDB, que é usado internamente para acessar o banco) e os schemas Pydantic (usados para validar o que entra e formatar o que sai da API) — são responsabilidades diferentes, mesmo representando o mesmo recurso.

4.7 Implementando GET e POST com o banco real (main.py)

```
# main.py
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session
from database import Base, engine, get_db
from models import ProdutoDB
from schemas import ProdutoCreate, ProdutoResponse

Base.metadata.create_all(bind=engine) # cria as tabelas, se ainda não existirem

app = FastAPI()

@app.get('/produtos', response_model=list[ProdutoResponse])
def listar_produtos(db: Session = Depends(get_db)):
    return db.query(ProdutoDB).all()

@app.post('/produtos', response_model=ProdutoResponse, status_code=201)
def criar_produto(produto: ProdutoCreate, db: Session = Depends(get_db)):
```

```
novo_produto = ProdutoDB(**produto.dict())
db.add(novo_produto)
db.commit()
db.refresh(novo_produto)
return novo_produto
```

Uso de **Depends(get_db)** — o mecanismo de injeção de dependências do FastAPI, que entrega uma sessão de banco pronta para uso em cada endpoint — e a sequência add / commit / refresh ao cadastrar um novo registro.

4.8 Testando e validando a persistência real

Executar `"uvicorn main:app --reload"`, cadastrar um produto pela documentação interativa (`/docs`) e, em seguida, encerrar o servidor (`Ctrl+C`) e executá-lo novamente. Diferentemente da versão com lista em memória, os dados cadastrados continuam disponíveis — pois agora estão persistidos de verdade no MySQL.

5. Atividade prática

- **GET /produtos/{id}** — retorna um único produto, consultando o banco com `db.query(ProdutoDB).filter(ProdutoDB.id == produto_id).first()`, ou erro 404 caso não exista.
- **DELETE /produtos/{id}** — remove um produto do banco de dados real.

O código deverá ser salvo para ser reaproveitado na próxima aula (parte 2), quando serão tratados o método PUT e a criação de um novo recurso com seu próprio modelo e tabela.